

briefly describe how the threads are generated for this code.

#pragma omp parallel for spawns a team of threads

By default, OpenMP only attaches to the immediately following loop (r = 1 to 5).

The outer loop has 5 iterations (r = 1, 2, 3, 4, 5) distributed among 4 threads:

```
└─$ ./barrier_test4
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 4, k = 9, thread # = 2
r = 4, k = 10, thread # = 2
r = 3, k = 1, thread # = 1
r = 3, k = 2, thread # = 1
r = 3, k = 3, thread # = 1
r = 3, k = 4, thread # = 1
r = 3, k = 5, thread # = 1
r = 3, k = 6, thread # = 1
r = 3, k = 7, thread # = 1
r = 3, k = 8, thread # = 1
r = 3, k = 9, thread # = 1
r = 3, k = 10, thread # = 1
r = 1, k = 1, thread # = 0
r = 1, k = 2, thread # = 0
r = 1, k = 3, thread # = 0
r = 1, k = 4, thread # = 0
r = 1, k = 5, thread # = 0
r = 1, k = 6, thread # = 0
r = 1, k = 7, thread # = 0
r = 1, k = 8, thread # = 0
r = 1, k = 9, thread # = 0
r = 1, k = 10, thread # = 0
r = 2, k = 1, thread # = 0
r = 2, k = 2, thread # = 0
r = 2, k = 3, thread # = 0
r = 2, k = 4, thread # = 0
r = 2, k = 5, thread # = 0
r = 2, k = 6, thread # = 0
r = 2, k = 7, thread # = 0
r = 2, k = 8, thread # = 0
```

When you replace the code below with `#pragma omp parallel for collapse(2)` how are the threads generated.

With collapse(2) 50 iterations 13 runs

```
└─$ ./barrier_test4
r = 3, k = 7, thread # = 2
r = 3, k = 8, thread # = 2
r = 3, k = 9, thread # = 2
r = 3, k = 10, thread # = 2
r = 4, k = 1, thread # = 2
r = 4, k = 2, thread # = 2
r = 4, k = 3, thread # = 2
r = 4, k = 4, thread # = 2
r = 4, k = 5, thread # = 2
r = 4, k = 6, thread # = 2
r = 4, k = 7, thread # = 2
r = 4, k = 8, thread # = 2
r = 2, k = 4, thread # = 1
r = 2, k = 5, thread # = 1
r = 2, k = 6, thread # = 1
```